

# Analysis: Bot Trust

*BAM! Robots gave us 6 extra seconds of cooperation. Good job, robots!*

-- Cave Johnson (Portal 2)

Hopefully your robots were more focused on teamwork than Cave Johnson's were, or you would never get all the buttons pressed. Just be glad there were no mashy spike plates or potatoes to contend with! Perhaps in the finals...

Anyway, if you think about this problem from the perspective of a single robot, the strategy should be pretty intuitive: always move towards the next button and then push it as soon as it comes up in the sequence.

So the most natural solution to this problem is a straight simulation:

- Keep track of which buttons have been pressed, and look ahead in the sequence to figure out which button is next for each robot.
- One second at a time, have each robot move towards its next button.
- Once it gets to the button, the robot should push it if it's next in the sequence, and just wait otherwise.

At most 100 buttons needs to be pressed altogether and the distance between buttons is at most 100, so this solution will run plenty fast.

If you get stuck on the implementation, remember that you can see other contestants' source code. There is no better way to learn!

# Analysis: Candy Splitting

The main step needed to solve this problem is to understand Patrick's strange addition algorithm. For example, can we describe what happens when Patrick adds up several numbers instead of just two? It turns out we can: we should write all numbers in binary, align them by their least significant bit, and write 1 in those positions where we have an *odd* number of 1 bits in the summands (the numbers being added).

Consider this example: suppose Patrick needs to add 5, 7 and 9 together. First, he adds up 5 and 7 by writing them in binary and adding digit-by-digit without carry, as described in the problem statement:

```
  101
+ 111
-----
  010
```

The result is 010 in binary, which is 2. Now, he adds up 2 and 9:

```
  0010
+ 1001
-----
  1011
```

The result is 1011 in binary, which is 11. It is most instructive to look at what happened to the least significant bit: after adding up two of the numbers, we had a 0 in the least significant bit since both of the summands had a 1 there. However, we have a 1 in the least significant bit of the overall result since the third number had a 1 there as well. It's not hard to see that this generalizes as described above: for any bit, it will be equal to 1 in the overall sum if and only if this bit is set to 1 in an odd number of summands.

Having established that, we can now understand Sean's task better. He needs to separate the given set of numbers into two parts in such a way that for every bit position, either:

- In both parts, an odd number of summands have this bit set to 1, so that the corresponding bit in the sum is 1 for both parts, or
- In both parts, an even number of summands have this bit set to 1, so that the corresponding bit in the sum is 0 for both parts.

But saying that we need two numbers to be either both odd, or both even, is equivalent to saying that their sum must be even!

That allows us to reformulate Sean's task simply as: he needs to separate the given set of numbers into two parts in such a way that for every bit position, an even number of summands have the bit set to 1 across both parts put together. Suddenly, we understand that this condition doesn't rely on the way we separate the numbers into two parts at all! Either it is true for every bit position that an even number of all summands have the bit set to 1, in which case any separation into two non-empty piles will make Patrick happy, or there is some bit which is 1 in an odd number of summands, in which case there's no way to make Patrick happy.

For example, suppose Sean has pieces of candy with values 5, 7, 9 and 11. If he separates them into 5 and 7 for himself, and 9 and 11 for Patrick, Patrick will add 5 and 7 as  $5+7=101_2+111_2=010_2=2$ , and 9 and 11 as  $9+11=1001_2+1011_2=0010_2=2$ , so Patrick is happy.

But even if Sean takes 7, 9 and 11 and leaves just 5 to Patrick, Patrick will add 7, 9 and 11 as  $7+9+11=0111_2+1001_2+1011_2=0101_2=5$ , so he is still happy! It's not difficult to verify that in all other cases Patrick is happy as well.

All the above reasoning can be made simpler if you notice that Patrick's strange addition is exactly [bitwise exclusive or](#). The condition for Patrick's happiness can be reformulated as the bitwise exclusive or of all candy values being equal to zero. In many programming languages, bitwise exclusive or is already built in with the "^" operator, making this very easy to check!

Now, how does the overall solution for the problem work? First, we need to check if Patrick will be happy - as shown above, this does not depend on Sean's piles. If not, then we just output "NO" for this testcase. If yes, then Sean should maximize his pile, and this is achieved by taking all pieces of candy except the one with the smallest value.

# Analysis: GoroSort

## The Solution

This problem is very mathematical in nature, requiring a lot of thought and only a little code to solve correctly. We put it as a problem on the Qualifying Round to give you something different to try without having to worry about time pressure or advancement. We hope you enjoyed it!

For an arbitrary array  $A$ , let  $n(A)$  be the number of elements that are not already in the correct position. It turns out that the answer you are looking for is simply  $n(A)$ . Once you realize this, it is easy to calculate with a simple loop, but how do you prove it?

## The Proof

Let's first show that the expected number of hits is never more than  $n(A)$ . Suppose Goro always holds down only those elements that are already in position, and then he randomly permutes the rest. Let  $x(A)$  be the expected number of hits required for him to sort  $A$  using this strategy.

**Lemma:**  $x(A) = n(A)$  for all  $A$ .

We prove this by induction on  $n(A)$ . If  $n(A) = 0$ , then the array is already sorted, and we're done. To set up the induction, let's suppose we have proven the lemma already for smaller values of  $n(A)$  and we are now trying to prove it for  $A$ . Let  $p_t$  be the probability that exactly  $t$  elements are still out of position after the first hit, and let  $x'_t$  be the expected number of hits required in this case. We make three observations:

- $p_0 * 0 + p_1 * 1 + p_2 * 2 + \dots + p_N * N = N - 1$ . In English, this is saying that the *expected* number of elements that are out of position after the first hit is exactly  $N - 1$ , or equivalently, the expected number of elements that are put into position by the first hit is exactly 1. This follows from "linearity of expectation": Goro is permuting  $N$  elements; each one has probability exactly  $1/N$  of ending up in the correct position, and hence, the expected number of elements that end up in the correct position is  $N * 1/N = 1$ .
- $x'_t = t$  for  $t \leq N - 1$ . This is true by the inductive hypothesis.
- $x'_N = x(A)$ . If no elements are put into the correct position by the first hit, then we will just randomly permute them all again in the next step, so nothing has changed, and hence  $x'_N = x(A)$ .

Now let's write down a formula for  $x(A)$ :

$$\begin{aligned} & 1 + p_0 * x'_0 + p_1 * x'_1 + \dots + p_N * x'_N \\ &= 1 + p_0 * 0 + p_1 * 1 + \dots + p_N * N + p_N * (x(A) - N) \\ &= N + p_N * (x(A) - N), \end{aligned}$$

which simplifies to  $(N - x(A)) * (1 - p_N) = 0$ . Since  $p_N < 1$ , we must have  $x(A) = N$ , and the lemma is proven.

To complete the proof, we need to calculate  $y(A)$ , the expected number of hits required for Goro to sort  $A$  if he uses the (still unknown) optimal strategy. Since  $y(A) \leq x(A)$  by definition, we have already proven  $y(A) \leq n(A)$ .

To conversely prove  $n(A) \leq y(A)$ , it would be nice to just extend the proof of the previous lemma. There is one big technical issue though: it is possible for  $n(A)$  to go up if Goro doesn't hold down enough elements, and so it is tricky to set up an induction on  $n(A)$ . We'll resolve this by having a separate proof for this part, and this time use a slightly different induction hypothesis. We can then follow the previous argument very closely.

**Lemma 2:** Let  $K$  be a non-negative integer. Then for any  $k \leq K$ , the statement  $y(A) = k$  is equivalent to the statement  $n(A) = k$ .

We will prove this by induction on  $K$ . Both  $y(A) = 0$  and  $n(A) = 0$  are equivalent to the array already being sorted, so the  $K = 0$  case is clear. Now, let's suppose we have proven the lemma for  $K$  already, and are trying to prove it for  $K+1$ . Choose  $A$  such that  $y(A)$  is the smallest possible value larger than  $K$  and consider the optimal strategy for Goro. Let  $T$  be the number of elements that are either (a) not in the correct position in  $A$ , or (b) permuted when Goro hits the table. Define  $p_i$  and  $x'_i$  as we did before. Note that  $T \geq n(A) \geq K+1$  by the inductive hypothesis.

As in the previous lemma, we can now prove the following:

- $p_0 * 0 + p_1 * 1 + p_2 * 2 + \dots + p_T * T \geq T - 1$ .
- $x'_i = i$  for  $i \leq K$ . This follows directly from the inductive hypothesis.
- $x'_i \geq y(A)$  for  $i > K$ . By the inductive hypothesis,  $n(A') > K$  implies  $y(A') > K$ , which then implies  $y(A') \geq y(A)$ .

As before, we can now write  $y(A)$  as

$$\begin{aligned} & 1 + p_0 * x'_0 + p_1 * x'_1 + \dots + p_T * x'_T \\ & \geq 1 + (p_0 * 0 + p_1 * 1 + \dots + p_T * T) + (p_{K+1} + p_{K+2} + \dots + p_T) * (y(A) - T) \\ & \geq T + (p_{K+1} + p_{K+2} + \dots + p_T) * (y(A) - T) \end{aligned}$$

which simplifies to  $(y(A) - T) * (1 - p_{K+1} - \dots - p_T) \geq 0$ . The second term has to be positive (if not, then  $y(A) \geq \min(x'_{K+1}, x'_{K+2}, \dots) + 1$ , which contradicts the third bullet point above is therefore impossible), so we must have  $y(A) \geq T \geq n(A) \geq K+1$ . Equality holds only if  $n(A) = K+1$ . The first lemma guarantees  $y(A) \leq x(A) \leq K+1$  in this case, and the proof is complete!

## Comments

- If you go over the proof carefully, you can see there are two things Goro needs to do in order to be optimal. (1) He needs to always hold down elements that are already in the correct position, and (2) he needs to ensure that for each element  $x$  that is permuted, the element in  $x$ 's correct position is also permuted. This means he actually has some choice about what to do.
- On a programming contest, of course you do not need to work through a formal proof to implement a correct solution. The best contestants can solve this kind of problem by looking at small examples to see a pattern, and then using intuitive reasoning to see what's going on without formalizing everything. Mastering this kind of reasoning is a difficult art though!

# Analysis: Magicka

This problem can be solved with a simulation. First, we have to remember what elements combine to make other elements. A map of some sort, like a hash map, is a great way of doing this. Next we have to track the opposed elements, remembering that one element can be opposed to multiple other elements; a set of pairs, while not particularly efficient for this purpose, will do the trick.

Finally, the simulation itself. For each character, first we check to see if it combines with the last item on the element list, and combine it if so. If it doesn't combine, then we iterate through the elements already in the list and see if it's opposed to any of them -- if so, we clear the list. Finally, if neither of those conditions was met, we append it to the list. Here is some Pythonesque pseudocode that solves the problem:

```
# Let combo_list contain all the combinations as 3-letter strs.
# Let opposed_list contain all the opposed elements as 2-letter strs.
# Let invoke be a str containing the elements to invoke.
combos = dict()
opposed = dict()
for x in combo_list:
    combos[x[0] + x[1]] = x[2]
    combos[x[1] + x[0]] = x[2]
for x in opposed_list:
    opposed.add(x[0] + x[1])
    opposed.add(x[1] + x[0])
# Now combos contains a mapping from each pair to the thing it
# creates. If one of the combinations was "ABC", then
# combos["AB"] = "C" and combos["BA"] = "C".
# opposed is filled in a similar way.

element_list = []
for element in invoke:
    # If element_list isn't empty, the last element might combine
    # with the element being invoked.
    if element_list:
        last_two = element_list[-1] + element
        if last_two in combos:
            element_list[-1] = combos[last_two]
            continue

    # Now we iterate through element_list to see if anything there
    # is opposed to the element being invoked.
    wipe_list = False
    for e in element_list:
        if (e + element) in opposed:
            wipe_list = True
    if wipe_list:
        element_list = []
        continue

    # There was no combination and no erasing: just append the
    # element to the list.
    element_list.append(element)
```